

AI for OS-Level Side-Channel Attack Detection and Mitigation in Linux Kernels

Name: [Your Name]

Roll No.: [Your Roll Number]

Report Format: IEEE-style section ordering

Abstract

Side-channel attacks exploit microarchitectural behavior rather than software permission violations, allowing adversaries to leak sensitive information through cache access patterns, speculative execution effects, and memory disturbance behavior. This report presents a final integrated Linux-oriented detection framework that uses hardware performance counters (HPCs) and machine learning to distinguish benign execution from cache-based attack activity. Raw traces were collected from Ubuntu 24.04 LTS and Fedora 40 using `perf stat` at a 50 ms sampling interval over six baseline events: cache-misses, cache-references, instructions, cycles, branches, and branch-misses. A hybrid ingestion pipeline normalizes the differing output formats produced by Ubuntu and Fedora. The strongest baseline result recorded during development is a Random Forest classifier trained on the Ubuntu dataset, achieving 99.9535% accuracy on a held-out test set of 30,104 samples, while the finalized project context reports peak accuracy of 99.98%, F1-score of 0.999, and cross-OS performance above 95%. The final notebook integrates feature engineering, exploratory data analysis, Random Forest, SVM, and CNN-LSTM evaluation, cross-environment comparison, automatic saving of figures and tables to `results/`, artifact export to `artifacts/`, and runtime deployment preparation. A companion real-time monitor script performs sliding-window inference over streamed HPC measurements on AWS/Linux environments.

Introduction

Microarchitectural side-channel attacks remain a serious systems security challenge because they leak information by observing shared hardware behavior rather than by directly breaking software isolation. Attacks such as Spectre, Meltdown, Prime+Probe, Flush+Reload, and Rowhammer have shown that secrets can be inferred from cache activity, speculative execution side effects, or memory disturbance patterns. This means that even a correctly functioning operating system kernel may fail to detect information leakage occurring at the processor microarchitecture level.

This project focuses on cache-based covert-channel and side-channel detection in Linux systems using hardware performance counters. The key idea is that attack activity produces measurable deviations in counters such as cache misses, cache references, cycles, and branch behavior. If these deviations can be learned reliably by machine learning models, then the resulting detector can serve as the basis for a deployable runtime security monitor. A major part of the work was not only model training, but also making the data itself available through self-extraction and cleaning when public datasets proved inadequate.

Related Work

Foundational research such as Spectre and Meltdown established that speculative and out-of-order execution can expose protected data through microarchitectural side effects. Cache-centric studies like

FLUSH+RELOAD and practical last-level cache attacks demonstrated that cache-eviction-based methods are realistic and measurable. Linux `perf` documentation and processor manuals justify the use of hardware counters as the telemetry source, while prior HPC-based detection work and deep-learning side-channel analysis research support the use of Random Forest, SVM, and CNN-LSTM models in this project.

Problem Description

The problem addressed in this work is the real-time detection of cache-based side-channel attack activity in Linux using hardware performance counters, while maintaining high accuracy, low false-positive rate, and low runtime overhead. Traditional OS protections do not directly observe microarchitectural leakage behavior. Static threshold-based counter monitoring is often too brittle because legitimate workloads may also exhibit bursts of cache or branch activity. Therefore, the system must learn the difference between benign HPC signatures and attack signatures under realistic operating conditions.

Methodology / Algorithm

1. Collect labeled benign and attack traces using Linux `perf stat` at 50 ms intervals.
2. Normalize Ubuntu and Fedora raw counter output into a common tabular schema.
3. Pivot vertical event logs into one row per timestamp with one column per HPC event.
4. Engineer additional ratio and temporal features such as IPC, cache miss rate, branch miss rate, rolling means, and first differences.
5. Train and compare Random Forest, SVM, and CNN-LSTM.
6. Export artifacts and apply sliding-window runtime monitoring on AWS/Linux.

```
Algorithm DetectSideChannel(HPC_stream, trained_model)
1. Read live HPC sample from collector
2. Append sample to sliding buffer
3. If window not complete, continue collecting
4. Rebuild engineered features matching notebook order
5. Apply saved classical model or neural model
6. Compute attack score
7. If score >= threshold, emit alert
8. Continue monitoring
```

System Design / Architecture

The implemented system uses a layered architecture: Linux `perf` acts as the collection layer; a hybrid parser normalizes interval logs; a feature engine constructs raw, ratio, difference, and rolling features; trained classifiers perform inference; and the runtime monitor emits alerts. The project workspace stores visual outputs in `results/` and deployment artifacts in `artifacts/`. A separate runtime monitor script loads the saved classical model and feature-column ordering for real-time inference.

Complexity Analysis

Component	Time Complexity	Space Complexity	Notes
Hybrid parsing	$O(N)$	$O(N)$	Each log line is processed once.
Feature engineering	$O(NF)$	$O(NF)$	F engineered attributes per sample.
Random Forest training	$O(T \cdot N \log N \cdot F_{\text{sub}})$	$O(T \cdot N)$	Useful for nonlinear classification and feature ranking.
Linear SVM training	Approx. $O(E \cdot N \cdot F)$	$O(F)$	Scales better for large tabular datasets.
CNN-LSTM training	High, sequence-dependent	High	Captures temporal structure more effectively.

Experimental Setup

Two Linux environments were used: Ubuntu 24.04 LTS as the primary baseline and Fedora 40 as the cross-environment validation environment. All data were collected with `Linux perf stat` at 50 ms intervals over six events: cache-misses, cache-references, instructions, cycles, branches, and branch-misses. Custom `victim.c` and `attacker.c` programs were used to generate benign and interference-heavy traces. The analysis stack uses Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn, and TensorFlow/Keras.

An important exploratory branch involved attempting to collect comparable telemetry on a Lenovo Android tablet. This effort exposed a major portability challenge: ARM/Android environments often restrict user-space access to hardware counters unless the kernel is modified or the device is rooted. Although this path did not yield deployable telemetry, it was an important research finding because it revealed practical “hardware siloing” beyond the x86/Linux server ecosystem.

Results and Analysis

Metric	Value	Interpretation
Peak Accuracy	99.98%	Best recorded end-state performance from project context.
Ubuntu RF Baseline	99.9535%	Directly documented benchmark in checked notebook-related assets.
Cross-OS Generalization	>95%	Ubuntu-trained model still detects Fedora attack traces effectively.
F1-score	0.999	Near-perfect balance of precision and recall.

The final notebook now includes implemented evaluation paths for Random Forest, SVM, and CNN-LSTM on Ubuntu and Fedora, cross-OS transfer evaluation, automatic saving of confusion matrices and EDA outputs into `results/`, and runtime-manifest generation for live deployment. From a progress-report perspective, this means the project matured from isolated experimental scripts into a reusable analysis and deployment-preparation workflow.

Challenges and Limitations

- The strongest directly visible benchmark in checked notebook assets is still the Ubuntu Random Forest baseline, even though the full comparison pipeline is implemented.
- The attack family remains narrower than a full industrial benchmark suite.
- Cross-architecture proof across Intel, AMD, and ARM is not shown in the current checked assets.
- Live overhead and false-positive measurements depend on executing the runtime monitor on AWS with mixed workloads.
- Evasion-resistance should only be claimed if supported by separate benchmark outputs.

Conclusion and Future Work

The project has progressed from raw trace extraction to a final integrated workflow that includes preprocessing, feature engineering, comparative modeling, saved artifacts, saved visual outputs, and runtime-monitor preparation. The strongest reported metrics show that side-channel signatures are stable enough to support near-perfect detection across Linux environments.

At implementation level, the checked assets support completion of Objectives 1 through 5: HPC feature engineering, dataset generation, comparative modeling, cross-OS validation, and runtime-monitor preparation. The overall methodology is feasible, logically structured, and directly relevant to the problem statement. Future work should expand processor coverage, attack diversity, mixed-workload runtime benchmarks, and adversarial-evasion studies.

References

1. Kocher, Paul, et al. "Spectre Attacks: Exploiting Speculative Execution." *Spectre Attack*, 2019, www.spectreattack.com/spectre.pdf.
2. Lipp, Moritz, et al. "Meltdown: Reading Kernel Memory from User Space." *Meltdown Attack*, 2018, meltdownattack.com/meltdown.pdf.
3. Yarom, Yuval, and Katrina Falkner. "FLUSH+RELOAD: a High Resolution, Low Noise, L3 Cache Side-Channel Attack." *23rd USENIX Security Symposium*, 2014, www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-yarom.pdf.
4. Liu, Fangfei, et al. "Last-Level Cache Side-Channel Attacks are Practical." *2015 IEEE Symposium on Security and Privacy*, 2015, eprint.iacr.org/2015/564.pdf.
5. "Linux Perf Event Features and Design." *perf.wiki.kernel.org*, perf.wiki.kernel.org/index.php/Main_Page.
6. Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.
7. Musleh, M., et al. "Detecting Cache-Based Side-Channel Attacks Using Hardware Performance Counters." *IEEE HOST*, 2018.
8. Benadjila, Ryad, et al. "A Survey of Deep Learning for Side-Channel Analysis." *arXiv*, 2020, arxiv.org/abs/1905.03164.
9. Tanenbaum, Andrew S., and Herbert Bos. *Modern Operating Systems*. 4th ed., Pearson.